CERN - HSF

GSoC 26: Evaluation Test

AI-Accelerated Reconstruction for the ATLAS Tile Calorimeter at the HL-LHC

Mentors:
Luca Fiorini
Fernando Carriò

Student:
Rubén Sierra
sierraserranoruben@gmail.com

March 9, 2026

Contents

1 Introduction to the Problem **2**

2 Filter Design **3**
2.1 Model 3
2.2 Computational Implementation and Vectorization 5

3 Results Obtained **6**
3.1 Exploratory Data Analysis 6
3.2 Matrix Conditioning and Filter Weights 7
3.3 Reconstruction Performance and Residual Analysis 7
3.4 Matrix Stabilization via Tikhonov Regularization 9

4 Conclusions and Future Outlook **11**

References **12**

A Appendix: Python Code **13**

1 Introduction to the Problem

The ATLAS experiment relies on the Tile Calorimeter (TileCal), the central hadronic calorimeter, to reconstruct hadronic jets and contribute significantly to the measurement of missing transverse energy. TileCal is a sampling calorimeter composed of steel absorbers and plastic scintillators. When charged particles traverse the detector, they produce light in the active scintillators. This light is delivered through wavelength shifting (WLS) fibres to Photomultiplier Tubes (PMTs), which convert it into an analog electronic pulse.

To process these measurements, the continuous analog waveform from the PMT is digitized at a frequency of 40 MHz. This yields one discrete digital sample every 25 nanoseconds, perfectly coinciding with the temporal spacing between consecutive proton bunch crossings (BC) within the collider.

The High-Luminosity Challenge (HL-LHC)

The High-Luminosity Large Hadron Collider (HL-LHC) upgrade will drastically increase the instantaneous luminosity of the collider. While beneficial for physics reach, this introduces extreme pile-up conditions, with the expected average number of interactions per bunch crossing reaching $\langle \mu \rangle = 200$.

Because the calorimeter signals span multiple 25 ns samples, the waveform generated by a particle in the target bunch crossing overlaps with the signals from adjacent collisions. This phenomenon is highly dependent on the collider's luminosity.

Consequently, the digitized samples are not isolated measurements; they are a complex superposition of the target physical signal, the intrinsic electronic noise of the calorimeter, and the out-of-time pile-up.

Objective of the Reconstruction Algorithm

The primary objective of the signal processing algorithm is to accurately reconstruct the physical energy amplitude, time, and pedestal of the signal from its digital samples.

The established approach is the Optimal Filtering (OF) algorithm, which operates on multiply-sampled signals. The OF method reconstructs the physical parameters through a linear combination of the digitized samples. The mathematical goal of this linear approach is to calculate a set of weights that minimizes the impact on the energy resolution caused by both the intrinsic electronic noise and the pile-up. Achieving this minimization strictly requires utilizing the known pulse shape and the noise covariance matrix.

2 Filter Design

2.1 Model

The reconstruction of the energy deposited in the calorimeter relies on a linear combination of eight digitized samples to estimate the true signal amplitude. Let the discrete sample array be indexed relative to the signal peak as $i \in [-4, 3]$. The reconstructed amplitude estimator $\hat{A}$ is defined as a linear mapping:

$$\hat{A} = \sum_{i=-4}^{3} w_i S_i \tag{1}$$

The digital samples acquired every 25 ns represent discrete snapshots of a continuous physical waveform. To analytically account for timing jitter between the digitizer clock and the true signal origin, the sampled signal S_i is modeled using a first-order Taylor expansion. This linearization disentangles the amplitude from the timing phase:

$$S_i \simeq p + Ag_i - A\tau g_i' + n_i \tag{2}$$

where the components of the model are defined as follows:

- S_i is the measured digital sample (in ADC counts) at index i.
- p is the baseline pedestal offset of the readout electronics.
- A is the true physical amplitude of the signal we aim to reconstruct.
- g_i is the normalized reference pulse shape evaluated at sample time i.
- τ is the small phase shift (timing jitter) between the digitizer clock and the true signal origin.
- g_i' is the time derivative of the normalized reference pulse shape at sample time i.
- n_i is the total background noise term, encompassing both thermal electronic noise and out-of-time pileup.

The fundamental objective of the filter is to provide an optimal estimate of the amplitude, $\hat{A}$, that tightly converges on the true physical amplitude A. Because the raw samples are contaminated by background interference, any linear combination of those samples will inherently propagate some statistical variance. The Optimal Filter framework seeks to minimize the variance of this amplitude estimator, which is defined mathematically as the quadratic form:

$$Var(\hat{A}) = w^T R w \tag{3}$$

Here, w represents the column vector of the eight filter weights, and R denotes the background noise covariance matrix. The inclusion of R is crucial for the physical accuracy of the model. In high-luminosity experimental conditions, the noise is not purely independent; rather, it possesses strong temporal correlations due to overlapping pulses from neighboring proton collisions. The matrix R captures this behavior, properly weighting

the independent thermal noise on the diagonal against the correlated pileup noise on the off-diagonal elements.

However, minimizing $w^T R w$ without boundaries would trivially drive all weights to zero. To ensure the filter extracts the correct physical quantities rather than simply silencing the output, the minimization is strictly bounded by three structural constraints enforced via Lagrange multipliers:

1. **Amplitude Unbiasedness**: This constraint ensures that if a pure, perfectly timed signal is passed through the filter, the output accurately reflects the true physical amplitude without arbitrary scaling.

$$\sum_{i=1}^{N} g_i w_i = 1 \implies w^T g = 1 \tag{4}$$

2. **Phase Invariance**: Because the true signal arrival time can naturally fluctuate, this constraint guarantees that the amplitude reconstruction remains immune to small phase shifts (τ). Mathematically, it orthogonalizes the amplitude measurement from the timing measurement.

$$\sum_{i=1}^{N} g'_i w_i = 0 \implies w^T g' = 0 \tag{5}$$

3. **Pedestal Suppression**: By forcing the sum of the weights to exactly zero, the filter becomes intrinsically immune to constant baseline fluctuations. Any flat pedestal offset multiplied by these weights will sum to zero, entirely eliminating the need to explicitly subtract a baseline value before applying the filter.

$$\sum_{i=1}^{N} w_i = 0 \implies w^T \vec{1} = 0 \tag{6}$$

To solve this constrained optimization problem algebraically, we construct a unified constraint matrix C composed of the three condition vectors:

$$C = \begin{bmatrix} g & g' & \vec{1} \end{bmatrix} \tag{7}$$

Applying the method of Lagrange multipliers to minimize the variance subject to these boundaries yields a direct analytical solution for the optimal amplitude weights:

$$w = R^{-1} C (C^T R^{-1} C)^{-1} \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \tag{8}$$

Furthermore, this matrix formulation naturally extends to the reconstruction of the timing phase, τ. By shifting the target vector in the constrained optimization, a complementary set of time weights, v, can be derived. These weights minimize the variance of the timing estimate while remaining orthogonal to the amplitude constraints:

$$v = R^{-1}C(C^T R^{-1} C)^{-1} \begin{bmatrix} 0 \\ -1 \\ 0 \end{bmatrix} \tag{9}$$

The final timing phase is then reconstructed by taking the ratio of the time-weighted samples to the amplitude-weighted samples, effectively normalizing out the signal scale: $\tau = \frac{v^T S}{w^T S}$.

The linear parameters extracted by the Optimal Filter are relative to the normalized reference pulse shape g. Therefore, to evaluate the final physical quantity, an empirical energy calibration factor k is required to map the dimensionless amplitude estimator $\hat{A}$ back to the true reconstructed energy E_{reco}. The complete linear energy reconstruction is defined as:

$$E_{reco} = k \sum_{i=-4}^{3} w_i S_i \tag{10}$$

2.2 Computational Implementation and Vectorization

Evaluating the Optimal Filter across datasets of simulated High-Luminosity events requires rigorous computational resource management. A naive software implementation relying on standard iterative loops in Python would introduce severe interpreter overhead, leading to prohibitive execution times and inefficient memory allocation.

To address this, the entire evaluation pipeline was designed using strict array vectorization paradigms via the `NumPy` library. By casting the data into contiguous memory blocks, operations are pushed down to highly optimized, pre-compiled C and Fortran linear algebra routines (such as BLAS and LAPACK).

Key vectorized implementations in the provided script include:

- **Data Ingestion:** The `load_shards` function constructs the 8-sample sliding window across the entire dataset simultaneously using `np.hstack` and `np.vstack`, completely avoiding event-by-event iteration.
- **Matrix Operations:** The filter weights and subsequent signal reconstructions ($\hat{A}$ and τ) are computed using batch matrix multiplications (the `@` operator), allowing the CPU hardware to process thousands of events in parallel.
- **Metric Evaluation:** Residual calculations and standardizations rely on array broadcasting, minimizing memory duplication and maximizing computational throughput.

3 Results Obtained

3.1 Exploratory Data Analysis

Before constructing the linear filter, an Exploratory Data Analysis (EDA) was performed to extract the intrinsic physical properties of the signal and the noise.

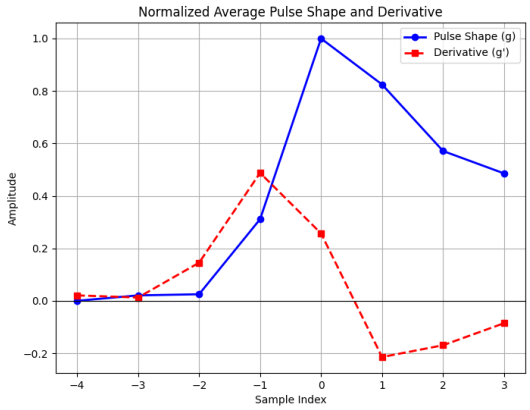

Figure 1: Average normalized pulse shape (g) and its derivative (g') extracted from high-energy events.

Figure 1 displays the normalized average pulse shape (g_i) of the calorimeter signal. Determining this underlying form function is a strict requirement for the Optimal Filter, as it acts as the primary constraint for the signal amplitude extraction.

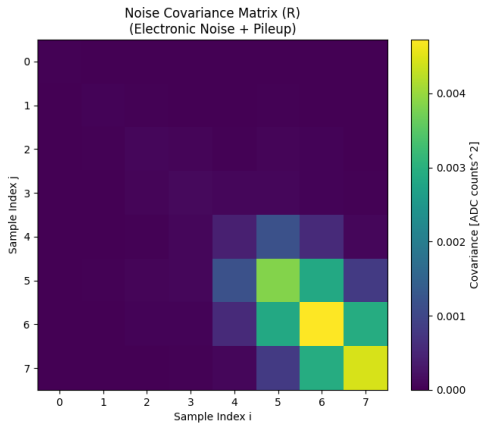

Figure 2: Noise covariance matrix (R) showcasing strong off-diagonal correlations.

Figure 2 visualizes the noise covariance matrix (R). The matrix is not strictly diagonal, which proves that the total background is not dominated by purely uncorrelated, thermal electronic noise. Instead, there are massive off-diagonal covariance elements. This strong temporal auto-correlation is the direct mathematical signature of out-of-time pileup. Because the TileCal pulse decay tail spans over 150 ns, the residual signals from overlapping proton interactions in preceding bunch crossings heavily correlate the consecutive 25 ns samples within the measurement window.

3.2 Matrix Conditioning and Filter Weights

The numerical stability of the noise covariance matrix R was subsequently analyzed. The calculated condition number of R is 15566.46. Because the out-of-time pile-up noise is highly auto-correlated, the rows and columns of R become nearly linearly dependent, resulting in a severely ill-conditioned matrix. This amplifies numerical instability when computing its inverse, R^{-1}, which is required for the analytical derivation of the weights.

Despite the ill-conditioned nature of the matrix, the Lagrange multiplier formulation successfully enforced the Optimal Filter boundary conditions up to machine precision. The constraint checks yielded:

- Amplitude Unbiasedness ($\sum w_i g_i = 1$): 0.9999999999999976
- Phase Invariance ($\sum w_i g'_i = 0$): -3.647×10^{-17}
- Pedestal Suppression ($\sum w_i = 0$):-1.388×10^{-16}

Applying the inverse covariance matrix resulted in the following 8-sample weight vector w:

$$w = [-0.2323, -0.3794, 0.6785, -1.4602, 1.9502, -0.7649, 0.2886, -0.0805]$$

The aggressive oscillation of these weights—particularly the sharp jump from -1.46 to $+1.95$ in adjacent samples—is a direct symptom of the ill-conditioned matrix inversion over-compensating for the correlated pile-up. The overall energy calibration constant mapping the ADC counts back to the physical scale was calculated as $k = 4900.5215$.

3.3 Reconstruction Performance and Residual Analysis

When the derived weights are applied to the validation and test datasets, the previously discussed mathematical instability propagates directly into the physical reconstruction metrics.

Metric	**Value**
Mean Relative Residual	-1.55987
RMS Relative Residual	30.23862
Mean Reconstructed Time	2.28146 BC
RMS Reconstructed Time	956.41596 BC

Table 1: Validation set reconstruction metrics.

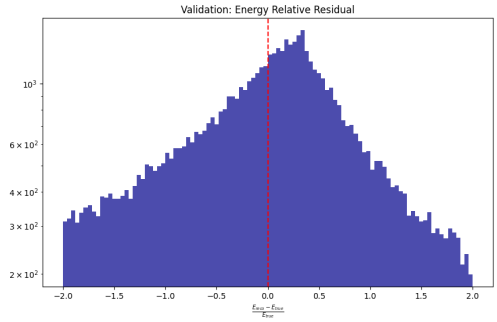

Figure 3: Validation set relative residual distribution.

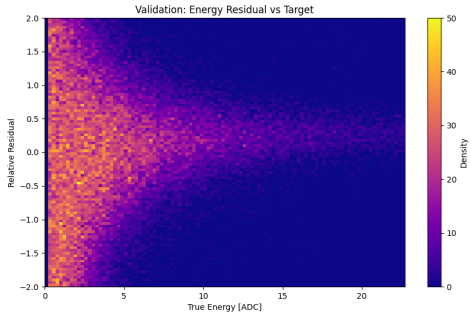

Figure 4: Validation relative residual vs. target signal.

Validation dataset Figures 3 and 4 illustrate the distribution of the relative residuals for the validation dataset. The logarithmic histogram reveals a highly broadened residual distribution. While the mean relative residual indicates a moderate systematic bias (~ -1.5), the Root Mean Square (RMS) reaches ~ 30, reflecting large fluctuations in the reconstruction.

The two–dimensional distribution in Figure 4 shows that the relative error varies significantly across the energy spectrum. At lower energies, the residual spread is strongly dominated by unsuppressed pile-up fluctuations, whereas at higher energies the proportionality between reconstructed and true signal is better preserved, although still affected by the skewed weights.

Metric	**Value**
Mean Relative Residual	-1.54247
RMS Relative Residual	25.52066
Mean Reconstructed Time	1.38252 BC
RMS Reconstructed Time	145.90370 BC

Table 2: Test set reconstruction metrics.

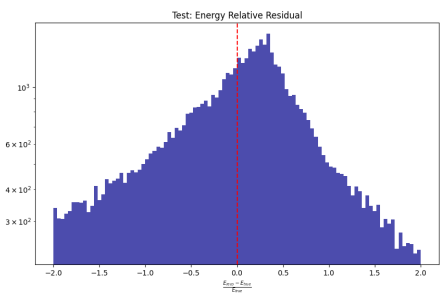

Figure 5: Test set relative residual distribution.

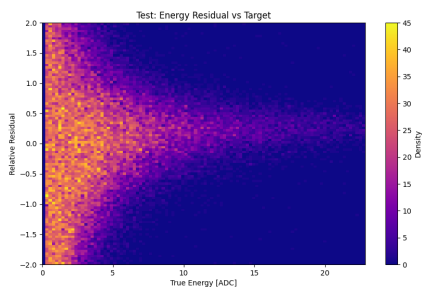

Figure 6: Test relative residual vs. target signal.

Test dataset The test dataset exhibits a similar behavior. As shown in Figures 5 and 6, the residual distributions remain broad, with an RMS of approximately 25. The

systematic bias is comparable to that observed in the validation dataset, indicating that the instability originates from the reconstruction method itself rather than from overfitting effects.

Timing reconstruction breakdown Crucially, the timing reconstruction mathematically collapses under these noise conditions.

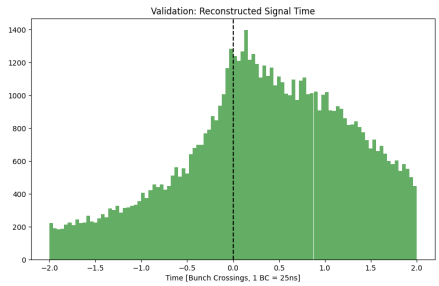

Figure 7: Validation set reconstructed time distribution.

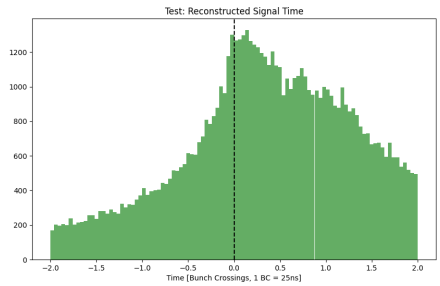

Figure 8: Test set reconstructed time distribution.

Figures 7 and 8 show that the distribution of reconstructed time parameters becomes completely unphysical. The RMS of the reconstructed time reaches hundreds of Bunch Crossings (BC).

This catastrophic failure is a direct consequence of the mathematical formulation $\tau = \frac{v^T S}{w^T S}$. In a high-pileup environment, the amplitude estimate $(w^T S)$ for background-dominated events frequently approaches zero due to the strict pedestal suppression constraint. Dividing the timing vector output by a near-zero amplitude causes the ratio to diverge toward infinity. This confirms that the severe correlations present in the pile-up noise render the strictly linear timing formula fragile and unreliable for extreme HL-LHC pile-up conditions.

3.4 Matrix Stabilization via Tikhonov Regularization

To address the numerical instability caused by the severely ill-conditioned noise covariance matrix, Tikhonov regularization (L2 penalty) was applied. This technique introduces a penalty term, λ, to the diagonal elements of the covariance matrix, defining a regularized matrix

$$R_{reg} = R + \lambda I \tag{11}$$

The objective is to stabilize the matrix inversion without excessively distorting the physical signal constraints.

A parameter space search was performed to identify the optimal regularization penalty that minimizes the relative residual Root Mean Square (RMS) on the validation dataset. As illustrated in Figure 9, the validation RMS exhibits a stable plateau before diverging rapidly for $\lambda > 10^{-6}$. The optimal penalty was identified as $\lambda = 8.38 \times 10^{-7}$.

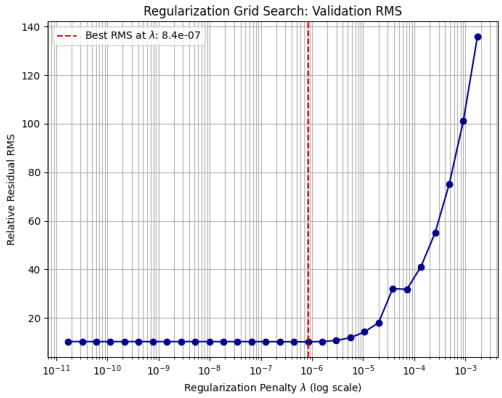

Figure 9: Validation RMS as a function of the regularization penalty λ.

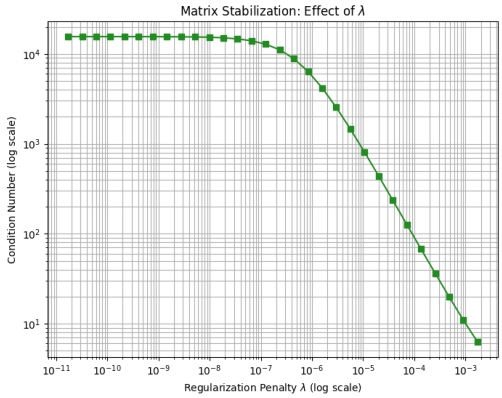

Figure 10: Matrix condition number as a function of the regularization penalty λ.

Figure 10 demonstrates the effect of this regularization on the matrix condition number. While increasing λ predictably forces the matrix toward a more stable mathematical regime, the optimal λ value only reduces the condition number to 6374.89. This represents a necessary compromise between mitigating numerical instability and preserving the underlying integrity of the physics model.

Despite the addition of the regularization term, the fundamental boundary constraints of the Optimal Filter are rigorously preserved up to machine precision:

- Amplitude Unbiasedness ($\sum w_i g_i = 1$): 0.9999999999999998
- Phase Invariance ($\sum w_i g_i' = 0$): -1.24×10^{-16}
- Pedestal Suppression ($\sum w_i = 0$): 1.30×10^{-15}

Metric	**Value**
Mean Relative Residual	-1.46197
RMS Relative Residual	26.10127
Mean Reconstructed Time	0.87487 BC
RMS Reconstructed Time	434.45644 BC

Table 3: Test set reconstruction metrics using Tikhonov Regularization.

However, applying these regularized weights to the test dataset yields limited practical improvement in the final reconstruction. The recalibrated evaluation (utilizing a regularized calibration constant $k = 5018.71$) results in an RMS Relative Residual of 26.10, as detailed in Table 3. While the condition number of the system was improved, the residual spread remains extremely broad. Furthermore, the timing reconstruction (τ) still exhibits an entirely unphysical RMS of 434.45 Bunch Crossings. This confirms that standard linear regularization techniques are insufficient to overcome the severe, highly correlated background interference inherent to High-Luminosity conditions.

4 Conclusions and Future Outlook

In this evaluation, the standard linear Optimal Filter (OF) algorithm was implemented and rigorously tested for signal reconstruction in the ATLAS Tile Calorimeter under simulated High-Luminosity LHC (HL-LHC) conditions.

The analysis revealed that the linear approach completely breaks down under extreme out-of-time pileup. While the Lagrange multiplier formulation mathematically enforces amplitude unbiasedness and pedestal suppression, the filter's core theoretical formulation relies on a first-order Taylor expansion that strictly assumes the presence of a single in-time signal pulse superimposed on zero-mean noise.

At the HL-LHC, this physical assumption is fundamentally violated. The long exponential decay tail of the calorimeter pulses creates a dynamic, continuous superposition of multiple overlapping interactions. This physical incompatibility mathematically manifests as a severely ill-conditioned noise covariance matrix ($R \approx 15500$). Consequently, the unregularized filter yields an entirely unphysical timing reconstruction RMS of over 950 Bunch Crossings and a massive spread in relative energy residuals.

Attempts to stabilize the linear model through Tikhonov regularization confirmed its theoretical limits. Although introducing an optimal L2 penalty ($\lambda = 8.38 \times 10^{-7}$) successfully reduced the matrix condition number to a mathematically manageable regime (6374.89), it yielded negligible improvements in physical accuracy. The failure of regularization to recover the timing reconstruction (RMS $>$ 400 BC) or narrow the residual spread proves that the problem is not merely numerical instability during matrix inversion, but the inherent inability of a static linear operator to resolve multiple overlapping signal templates.

The Necessity for AI-Accelerated Reconstruction

It is concluded that variance minimization via a linear L_2 norm is statistically insufficient for the HL-LHC. Because high-luminosity pileup introduces multiple pulses and highly skewed, non-Gaussian background distributions, advanced models capable of learning complex temporal representations are required.

To overcome these theoretical limits, a transition toward Deep Learning-based reconstruction algorithms is necessary. Architectures such as 1D Convolutional Neural Networks (1D-CNNs) are naturally suited to perform multi-pulse deconvolution directly from the raw waveform data. To operate within TileCal's strict front-end electronic constraints, future implementations must focus on compact ML models synthesizable onto Field Programmable Gate Arrays (FPGAs), ensuring the sub-microsecond inference latency and resource efficiency required for real-time physics trigger decisions.

References

[1] E. Fullana, et al. *Optimal Filtering in the ATLAS Hadronic Tile Calorimeter*. ATL-TILECAL-2005-001, CERN, 2005.

[2] F. Curcio. *Machine Learning-Based Energy Reconstruction for the ATLAS Tile Calorimeter at HL-LHC*. 2nd EuCAIFCon, ATLAS Collaboration, 2025.

[3] D. Oliveira Gonçalves. *Energy reconstruction of the ATLAS Tile Calorimeter under high pile-up conditions using the Wiener filter*. J. Phys.: Conf. Ser. 1525 012092, 2020.

A Appendix: Python Code

```python
import glob
import os

import matplotlib
import matplotlib.pyplot as plt
import numpy as np
import torch

# Use a non-interactive backend to avoid requiring a display/GUI
matplotlib.use("Agg")


def load_shards(folder_path):
    """
    Use:
        Reads .pt data files and creates a sliding window of signal samples.
        It shifts data to include a historical sample (n-1), creating an
        8-sample snapshot for each signal event.
    Args:
        folder_path (str): Path to the directory containing PyTorch shard files.
    Returns:
        tuple: (x_data, y_data)
            - x_data (np.ndarray): 2D array (Samples, 8) of signal windows.
            - y_data (np.ndarray): 1D array of target energy values.
    """
    files = sorted(glob.glob(os.path.join(folder_path, '*.pt')))
    x_final_list, y_final_list = [], []

    for f in files:
        # Load shard data onto CPU
        data = torch.load(f, map_location='cpu', weights_only=False)
        x_np = data['X'].numpy()[:, 1, :]  # Shape (M, 7) - Low Gain Channel
        y_np = data['y'].numpy()[:, 1]     # Shape (M,) - Low Gain Target

        # Vectorized window construction:
        # We need sample [n-4...n+3]. The shard provides [n-3...n+3].
        # 'prefix' takes the first sample of the previous row to act as n-4.
        prefix = x_np[:-1, 0:1]
        current_windows = x_np[1:, :]

        # Horizontal stack to create the 8-sample window
        windows_8 = np.hstack((prefix, current_windows))

        x_final_list.append(windows_8)
        y_final_list.append(y_np[1:])

    # combine all shards into single arrays
    return np.vstack(x_final_list), np.concatenate(y_final_list)


def eda(x_train, y_train):
```

```python
    """
    Use:
        Exploratory Data Analysis. Isolates high-energy pulses to find the
        average pulse shape (g) and low-energy pulses to calculate the
        noise covariance matrix (R).
    Args:
        x_train (np.ndarray): Input signal windows.
        y_train (np.ndarray): True energy targets.
    Returns:
        tuple: (g_shape, g_prime, noise_cov)
            - g_shape (np.ndarray): Normalized average pulse profile.
            - g_prime (np.ndarray): Derivative of the pulse profile.
            - noise_cov (np.ndarray): Statistical noise covariance matrix.
    """
    # Use top 2% energy events to define the ideal pulse shape
    high_energy_mask = y_train > np.percentile(y_train, 98)
    x_high = x_train[high_energy_mask]

    # Baseline subtraction and normalization
    g_shape = np.mean(x_high - x_high[:, 0:1], axis=0)
    g_shape /= np.max(g_shape)
    g_prime = np.gradient(g_shape)

    # Visualization of Signal Pulse
    plt.figure(figsize=(8, 6))
    sample_indices = np.arange(-4, 4)
    plt.plot(sample_indices, g_shape, marker='o', label='Pulse Shape (g)',
             color='blue', linewidth=2)
    plt.plot(sample_indices, g_prime, marker='s', linestyle='--',
             label="Derivative (g')", color='red', linewidth=2)
    plt.axhline(0, color='black', linewidth=0.8)
    plt.title("Normalized Average Pulse Shape and Derivative")
    plt.xlabel("Sample Index")
    plt.ylabel("Amplitude")
    plt.legend()
    plt.grid(True)
    plt.savefig("Images/eda_pulse_shape.png")
    plt.close()

    # Use bottom 10% energy events to characterize electronic noise
    low_energy_mask = y_train < np.percentile(y_train, 10)
    x_low = x_train[low_energy_mask]
    noise_cov = np.cov(x_low.T)

    # Visualization of Noise Correlation
    plt.figure(figsize=(8, 6))
    plt.imshow(noise_cov, cmap='viridis', interpolation='nearest')
    plt.colorbar(label='Covariance [ADC counts^2]')
    plt.title("Noise Covariance Matrix (R)\n(Electronic Noise + Pileup)")
    plt.xlabel("Sample Index i")
    plt.ylabel("Sample Index j")
    plt.savefig("Images/noise_covariance_matrix.png")
    plt.close()

    return g_shape, g_prime, noise_cov


def calculate_of3_weights(g, g_prime, noise_r):
```

```python
    """
    Use:
        Derives Optimal Filter weights by solving the constrained
        minimization problem: Minimize variance subject to
        reconstructing amplitude, timing drift, and pedestal.
    Args:
        g (np.ndarray): Reference pulse shape.
        g_prime (np.ndarray): Pulse shape derivative.
        noise_r (np.ndarray): Noise covariance matrix.
    Returns:
        tuple: (w, v)
            - w (np.ndarray): Energy reconstruction weights.
            - v (np.ndarray): Timing reconstruction weights.
    """
    # C matrix contains constraints for [Amplitude, Time, Pedestal]
    r_inv = np.linalg.inv(noise_r)
    c_mat = np.vstack((g, g_prime, np.ones_like(g))).T
    m_mat = np.linalg.inv(c_mat.T @ r_inv @ c_mat)

    # Extract weights for Energy (w) and Time (v)
    w_weights = r_inv @ c_mat @ m_mat @ np.array([1.0, 0.0, 0.0])
    v_weights = r_inv @ c_mat @ m_mat @ np.array([0.0, -1.0, 0.0])

    print("Condition Number of R:", np.linalg.cond(noise_r))
    print("Constraint g: ", w_weights @ g)      # Should be approx 1.0
    print("Constraint g': ", w_weights @ g_prime) # Should be approx 0.0
    print("Constraint 1: ", w_weights @ np.ones_like(g)) # Should be approx 0.0

    return w_weights, v_weights


def invert_standardization(y_scaled, mean, std):
    """
    Use:
        Converts normalized/standardized energy targets back into
        physical units (ADC counts).
    Args:
        y_scaled (np.ndarray): Normalized data.
        mean (float): Mean used for original scaling.
        std (float): Std dev used for original scaling.
    Returns:
        np.ndarray: Physical scale data.
    """
    return (y_scaled * std) + mean


def evaluate_metrics(y_true, y_pred, t_pred, dataset_name):
    """
    Use:
        Calculates residuals and creates histograms for Energy and Time
        reconstruction accuracy.
    Args:
        y_true, y_pred (np.ndarray): True vs Predicted physical energy.
        t_pred (np.ndarray): Predicted signal time.
        dataset_name (str): Label for saved plots.
    """
    # Filter out pure noise for metric calculation
```

```python
    mask = y_true > 0.1
    y_t, y_p, t_p = y_true[mask], y_pred[mask], t_pred[mask]

    # Relative Error
    residuals = (y_p - y_t) / y_t
    mean_res, rms_res = np.mean(residuals), np.std(residuals)

    print(f"\n--- {dataset_name} Energy Metrics ---")
    print(f"Mean Relative Residual: {mean_res:.5f}")
    print(f"RMS Relative Residual:  {rms_res:.5f}")
    print(f"Mean Reconstructed Time: {np.mean(t_p):.5f} BC")
    print(f"RMS Reconstructed Time:  {np.std(t_p):.5f} BC")

    # Plotting Energy Histogram
    plt.figure(figsize=(10, 6))
    plt.hist(residuals, bins=100, range=(-2, 2), color='darkblue', alpha=0.7)
    plt.axvline(0, color='red', linestyle='--')
    plt.title(f"{dataset_name}: Energy Relative Residual")
    plt.xlabel(r"$\frac{E_{reco} - E_{true}}{E_{true}}$")
    plt.yscale('log')
    plt.savefig(f"Images/{dataset_name}_energy_residual_hist.png")

    # Plotting Energy Correlation (2D)
    plt.figure(figsize=(10, 6))
    plt.hist2d(y_t, residuals, bins=[100, 100],
               range=[[0, np.percentile(y_t, 99)], [-2, 2]], cmap='plasma')
    plt.colorbar(label='Density')
    plt.title(f"{dataset_name}: Energy Residual vs Target")
    plt.xlabel("True Energy [ADC]")
    plt.ylabel("Relative Residual")
    plt.savefig(f"Images/{dataset_name}_energy_residual_2d.png")

    # Plotting Time Distribution
    plt.figure(figsize=(10, 6))
    plt.hist(t_p, bins=100, range=(-2, 2), color='forestgreen', alpha=0.7)
    plt.axvline(0, color='black', linestyle='--')
    plt.title(f"{dataset_name}: Reconstructed Signal Time")
    plt.xlabel("Time [Bunch Crossings, 1 BC = 25ns]")
    plt.savefig(f"Images/{dataset_name}_time_distribution.png")

    plt.close('all')


def study_tikhonov_grid_search(g, noise_r, x_val, y_val_true):
    """
    Use:
        Finds the optimal Tikhonov regularization parameter (lambda) by
        testing a range of values and measuring the resulting RMS error
        on a validation set.
    Args:
        g, noise_r: Signal shape and unregularized noise matrix.
        x_val, y_val_true: Data used for tuning.
    Returns:
        float: The lambda value that minimizes residual RMS.
    """
    base_thermal_variance = np.mean(np.diag(noise_r))
    # Define Test range
```

```python
    lambdas_tested = np.logspace(-8, 0, num=30) * base_thermal_variance

    conds, valid_rms = [], []
    mask_physical = y_val_true > 0.1
    y_target_evaluate = y_val_true[mask_physical]

    for lam in lambdas_tested:
        # Apply Tikhonov regularization: R_reg = R + lambda*I
        r_reg = noise_r + lam * np.eye(noise_r.shape[0])
        conds.append(np.linalg.cond(r_reg))

        r_inv = np.linalg.inv(r_reg)
        w_curr = (r_inv @ g) / (g.T @ r_inv @ g + 1e-12)

        preds_raw = x_val @ w_curr

        # Calibration for the current lambda
        cal_m = y_val_true > np.percentile(y_val_true, 90)
        if np.any(preds_raw[cal_m] > 1.0):
            k_curr = np.mean(y_val_true[cal_m] / preds_raw[cal_m])
        else:
            k_curr = 1.0

        y_p = preds_raw[mask_physical] * k_curr
        relative_residuals = (y_p - y_target_evaluate) / y_target_evaluate
        valid_rms.append(np.std(relative_residuals))

    best_idx = np.argmin(valid_rms)
    best_lam = lambdas_tested[best_idx]

    # Save Study Plot (RMS vs Lambda)
    plt.figure(figsize=(8, 6))
    plt.semilogx(lambdas_tested, valid_rms, marker='o', color='darkblue')
    plt.axvline(best_lam, color='red', linestyle='--',
                label=f'Best RMS at $\\lambda$: {best_lam:.1e}')
    plt.title("Regularization Grid Search: Validation RMS")
    plt.xlabel("Regularization Penalty $\\lambda$ (log scale)")
    plt.ylabel("Relative Residual RMS")
    plt.grid(True, which="both")
    plt.legend()
    plt.savefig("Images/study_lambda_rms.png")
    plt.close()

    # Save Study Plot (Condition Number vs Lambda)
    plt.figure(figsize=(8, 6))
    plt.loglog(lambdas_tested, conds, marker='s', color='forestgreen')
    plt.title("Matrix Stabilization: Effect of $\\lambda$")
    plt.xlabel("Regularization Penalty $\\lambda$ (log scale)")
    plt.ylabel("Condition Number (log scale)")
    plt.grid(True, which="both")
    plt.savefig("Images/study_lambda_cond.png")
    plt.close()

    print(f"Optimal Lambda: {best_lam:.2e} | Condition R_reg: {conds[best_idx]:.2f}")
    return best_lam
```

```python
if __name__ == "__main__":
    # Create directory for plots if it doesn't exist
    os.makedirs("Images", exist_ok=True)

    # Load Data
    X_TRAIN, Y_TRAIN = load_shards('data/train/')
    X_VAL, Y_VAL = load_shards('data/val/')
    X_TEST, Y_TEST = load_shards('data/test/')

    # Load Stats for Target Re-scaling
    stats = np.load('data/y_stats.npz')
    y_mean = stats['mean'].flatten()[1]
    y_std = stats['std'].flatten()[1]

    print("--- STANDARD OPTIMAL FILTER ---")

    # Model Design
    g_vec, gp_vec, r_mat = eda(X_TRAIN, Y_TRAIN)
    w_final, v_final = calculate_of3_weights(g_vec, gp_vec, r_mat)

    # Calibration (Determine the k-factor to map weights to physical units)
    y_tr_true = invert_standardization(Y_TRAIN, y_mean, y_std)
    y_tr_reco_raw = X_TRAIN @ w_final
    cal_mask = y_tr_true > np.percentile(y_tr_true, 90)
    k_const = np.mean(y_tr_true[cal_mask] / y_tr_reco_raw[cal_mask])

    print(f"Calibration Constant (k): {k_const:.4f}")

    # Evaluate Standard Filter
    eval_list = [("Validation", X_VAL, Y_VAL), ("Test", X_TEST, Y_TEST)]
    for name, x_data, y_raw in eval_list:
        y_true_vals = invert_standardization(y_raw, y_mean, y_std)
        e_raw = x_data @ w_final
        e_reco = e_raw * k_const
        tau_reco = (x_data @ v_final) / (e_raw + 1e-9)

        evaluate_metrics(y_true_vals, e_reco, tau_reco, name)

    print("\n--- OPTIMAL FILTER WITH TIKHONOV REGULARIZATION ---")

    # Grid Search for Stabilization
    Y_VAL_TRUE = invert_standardization(Y_VAL, y_mean, y_std)
    lambda_opt = study_tikhonov_grid_search(g_vec, r_mat, X_VAL, Y_VAL_TRUE
    )

    # Apply Regularization to the Noise Matrix
    r_mat_reg = r_mat + lambda_opt * np.eye(r_mat.shape[0])

    w_reg, v_reg = calculate_of3_weights(g_vec, gp_vec, r_mat_reg)

    # Final Regularized Evaluation
    y_true_test = invert_standardization(Y_TEST, y_mean, y_std)
    e_raw_reg = X_TEST @ w_reg

    # Recalculate k specifically for regularized weights
    cal_mask_eval = y_true_test > np.percentile(y_true_test, 90)
    k_eval = np.mean(y_true_test[cal_mask_eval] / e_raw_reg[cal_mask_eval])
    print(f"Regularized Calibration Constant (k): {k_eval:.4f}")
```

```

    e_reco_reg = e_raw_reg * k_eval
    tau_reco_reg = (X_TEST @ v_reg) / (e_raw_reg + 1e-9)

    evaluate_metrics(y_true_test, e_reco_reg, tau_reco_reg, "
   Test_Regularized")
```

Listing 1: Signal Processing Script